

Practical 05: ERP data analysis

```
In [16]: 1 # Credits: this notebook was written by Ilaria Gavetti for the Cognitive Neuroscience Executive Functio
          2
          3 # MNE-toolbox was used as reference to develop and implement the code.
```

```
! pip install mne
```

```
In [1]: 1 import mne
          2 import os
          3 import numpy as np
          4 import csv
          5 import pandas as pd
```

Welcome to the data-analysis part of the Executive Functions practical. At this point, you should have:

1. collected your data
2. received an output csv file from the data collection procedure, ...

Follow the instructions below to properly upload and conduct your data-analysis:

Step 0: upload the files

First, I suggest creating a "base" directory, called "Executive functions practical".

Then, please upload all the .csv files from your laptop in the directory **"../Executive functions practical/1. Raw files"**, where the **"..."** part is your personal path to get to **"Executive functions practical"** and **1. Raw files** is a folder in this directory which you should manually add.

Also, since we will be saving intermediate files during the analysis, add the following folders to your **"Executive functions practical"** directory:

2. Loading & Digitization

3. ICA

4. Epoch (with inside 1 other folder called **Annotated**)

The code you will run already has these directories implemented.

Last, but not least, change your current working directory to **"Executive functions practical"**, to ensure the code runs smoothly.

Step 1: Loading

```
In [2]: 1 os.getcwd()
```

```
'C:\\Users\\IamAh\\Desktop\\UNIVERSITY - CSAI\\Cognitive Neuroscience\\Practical_05\\Executive functions practical'
```

```
In [8]: 1 # Read the CSV file as a NumPy array
2
3 for file in os.listdir("1. Raw Files"):
4     print(file)
5     if file != '.ipynb_checkpoints':
6         if "EEG" in file:
7             print(file)
8             data = pd.read_csv("1. Raw Files/"+file, delimiter=",", verbose = True)
9             data_sub = data[["FZ", "C3", "CZ", "C4", "PZ", "P07", "OZ", "P08"]] # only select the elec
10            # Print file name
11            print("****"+file+"****")
12        elif "Unicorn" in file:
13            print(file)
14            data = pd.read_csv("1. Raw Files/"+file, delimiter=",", verbose = True)
15            data_sub = data[["EEG 1", "EEG 2", "EEG 3", "EEG 4", "EEG 5", "EEG 6", "EEG 7", "EEG 8"]] #
16            # Print file name
17            print("****"+file+"****")
18
19 # Some information about the channels
20 ch_names_new = ["Fz", "C3", "Cz", "C4", "Pz", "P07", "Oz", "P08"] # from Unicorn BCI manual
21
22 # Sampling rate of the Unicorn BCI
23 sfreq = 250 # Hz
24
25 # Create the info structure needed by MNE
26 info = mne.create_info(ch_names_new, sfreq, ch_types = ["eeg"]*len(ch_names_new), verbose = Tru
27
28 # Set montage
29 info.set_montage(montage = 'standard_1020')
30
31
32 # Finally, create the Raw object
33 raw = mne.io.RawArray(data_sub.T, info, verbose = True)
34
35 # Filter
36 raw.filter(1,30)
37
38 # Plot it!
39 raw.plot(scalings="auto", duration = 5)
40 print("End")
41
42 # Save it!
43 raw.save("2. Loading & Digitization/"+file[0:19]+"_raw.fif", overwrite = True)
44
```

```
EEG_CleanStream_09h_57m_17s.csv
EEG_CleanStream_09h_57m_17s.csv
Tokenization took: 59.97 ms
Type conversion took: 31.25 ms
Parser memory cleanup took: 0.00 ms
Tokenization took: 31.70 ms
Type conversion took: 15.66 ms
Parser memory cleanup took: 0.00 ms
****EEG_CleanStream_09h_57m_17s.csv****
Creating RawArray with float64 data, n_channels=8, n_times=57255
Range : 0 ... 57254 = 0.000 ... 229.016 secs
Ready.
Filtering raw data in 1 contiguous segment
Setting up band-pass filter from 1 - 30 Hz
```

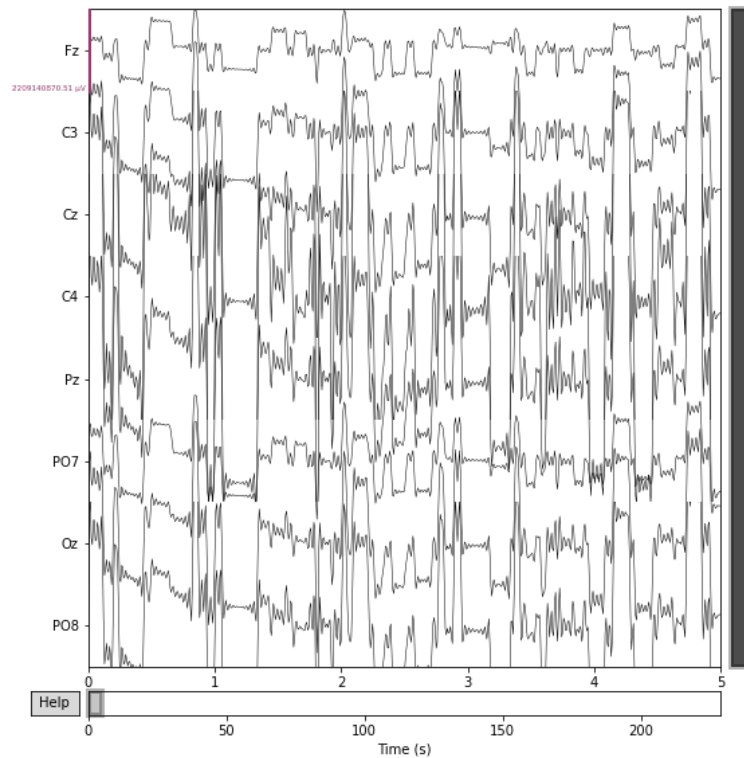
FIR filter parameters

```
-----
Designing a one-pass, zero-phase, non-causal bandpass filter:
- Windowed time-domain design (firwin) method
- Hamming window with 0.0194 passband ripple and 53 dB stopband attenuation
- Lower passband edge: 1.00
- Lower transition bandwidth: 1.00 Hz (-6 dB cutoff frequency: 0.50 Hz)
- Upper passband edge: 30.00 Hz
```

```
- Upper transition bandwidth: 7.50 Hz (-6 dB cutoff frequency: 33.75 Hz)
- Filter length: 825 samples (3.300 sec)
```

```
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 2 out of 2 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 3 out of 3 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 4 out of 4 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 8 out of 8 | elapsed: 0.0s finished
```

Using matplotlib as 2D backend.



```
End
Writing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_05\Executive functions practical\2. Loading & Digitizati
on\EEG_CleanStream_09h_raw.fif
Closing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_05\Executive functions practical\2. Loading & Digitizati
on\EEG_CleanStream_09h_raw.fif
[done]
```

Step 2: Adding annotations manually

```
In [9]: 1 from datetime import datetime, timezone
        2 from datetime import timedelta
```

```
In [10]: 1 os.getcwd()
```

```
'C:\\Users\\IamAh\\Desktop\\UNIVERSITY - CSAI\\Cognitive Neuroscience\\Practical_05\\Executive functions practical'
```

Add the name of the annotations file instead of '...' in all the cells below

```
In [13]: 1 np.load("2. Loading & Digitization/clean_annotations_array_09h_57m_19s.npy")

array([[ 'go', '1015'],
       [ 'no-go', '2140'],
       [ 'go', '3266'],
       [ 'go', '4392'],
       [ 'go', '5519'],
       [ 'go', '6645'],
       [ 'go', '7771'],
       [ 'go', '8897'],
       [ 'go', '10022'],
       [ 'go', '11147'],
       [ 'go', '12274'],
       [ 'go', '13400'],
       [ 'go', '14525'],
       [ 'go', '15651'],
       [ 'go', '16776'],
       [ 'no-go', '17902'],
       [ 'go', '19029'],
       [ 'no-go', '20155'],
       [ 'no-go', '21281'],
       [ 'go', '22406'],
       [ 'no-go', '23531'],
       [ 'no-go', '24656'],
       [ 'go', '25782'],
       [ 'no-go', '26908'],
       [ 'no-go', '28033'],
       [ 'no-go', '29159'],
       [ 'no-go', '30286'],
       [ 'no-go', '31411'],
       [ 'no-go', '32536'],
       [ 'no-go', '33661'],
       [ 'no-go', '34786'],
       [ 'go', '35912'],
       [ 'go', '37037'],
       [ 'go', '38163'],
       [ 'go', '39288'],
       [ 'go', '40415'],
       [ 'go', '41540'],
       [ 'go', '42666'],
       [ 'go', '43791'],
       [ 'go', '44916'],
       [ 'go', '46043'],
       [ 'go', '47169'],
       [ 'go', '48294'],
       [ 'go', '49419'],
       [ 'go', '50545'],
       [ 'go', '51670'],
       [ 'go', '52796'],
       [ 'go', '53922'],
       [ 'go', '55048'],
       [ 'no-go', '56174']], dtype='<U11')
```

Up until now, the raw data does not contain any markers, meaning that each file is a continuous flow of EEG data without any marker indicating where the non-oddball / oddball trials happen.

Nevertheless, we do know at what time from the recording onset each trial happens, so we need to make sure we annotate the EEG data with markers that reflect the information given in the **array**.

For example, if the first line of the **array** says " go - 1000 ", this means that the 1000th sample (an normal trial) was presented $4 \times 1000 = 4000$ ms after the recording onset, and so on.

In the next cell, we make sure that the EEG data is annotated such that the program will know at what point of the continuous EEG data each trial happens, and how long each trial lasts until the onset of a new trial.

Annotating will allow us to "Epoch" the data, which will make our life easier later in the notebook!

```
In [14]: 1 files = os.listdir("2. Loading & Digitization")
```

```
In [16]: 1 for file in files:
2         if file != '.ipynb_checkpoints_raw.fif' and "raw" in file:
3             if "EEG" in file:
4                 new_time = datetime(2023, 1, 27, hour=11, minute=59, second=20, microsecond=0, tzinfo=tzinfo=timezone.utc)
5             elif "Unicorn" in file:
6                 new_time = datetime(2023, 1, 27, hour=11, minute=59, second=20, microsecond=0, tzinfo=tzinfo=timezone.utc)
7             sample_data_raw_file = "2. Loading & Digitization/"+file
8             raw = mne.io.read_raw_fif(sample_data_raw_file, preload= True)
9             raw.set_meas_date(new_time)
10            meas_date = raw.info["meas_date"]
11            print(meas_date)
12
13            raw.plot(scalings = "auto", duration = 5)
14
15            # Annotation
16            array = np.load("2. Loading & Digitization/clean_annotations_array_09h_57m_19s.npy")
17
18            description = []
19            onset = []
20            start = float(0)
21
22            for i in range(array.shape[0]):
23                description.append(array[i][0])
24                onset.append(start)
25
26                # Increase
27                start = start + 4.5
28
29
30            time_format = '%Y-%m-%d %H:%M:%S.%f'
31            orig_time = (meas_date + timedelta(seconds=4)).strftime(time_format) # onset of annotations is
32
33            later_annot = mne.Annotations(onset=onset, duration=[0.5]*50, description=description, orig_time=orig_time)
34
35            raw.set_annotations(later_annot)
36            print(description)
37            print("Annotations onsets:", raw.annotations.onset)
38            print(meas_date, orig_time)
39
40
41            raw.save("4. Epoch/Annotated files/"+file[0:19]+"_annot_raw.fif", overwrite = True)
```

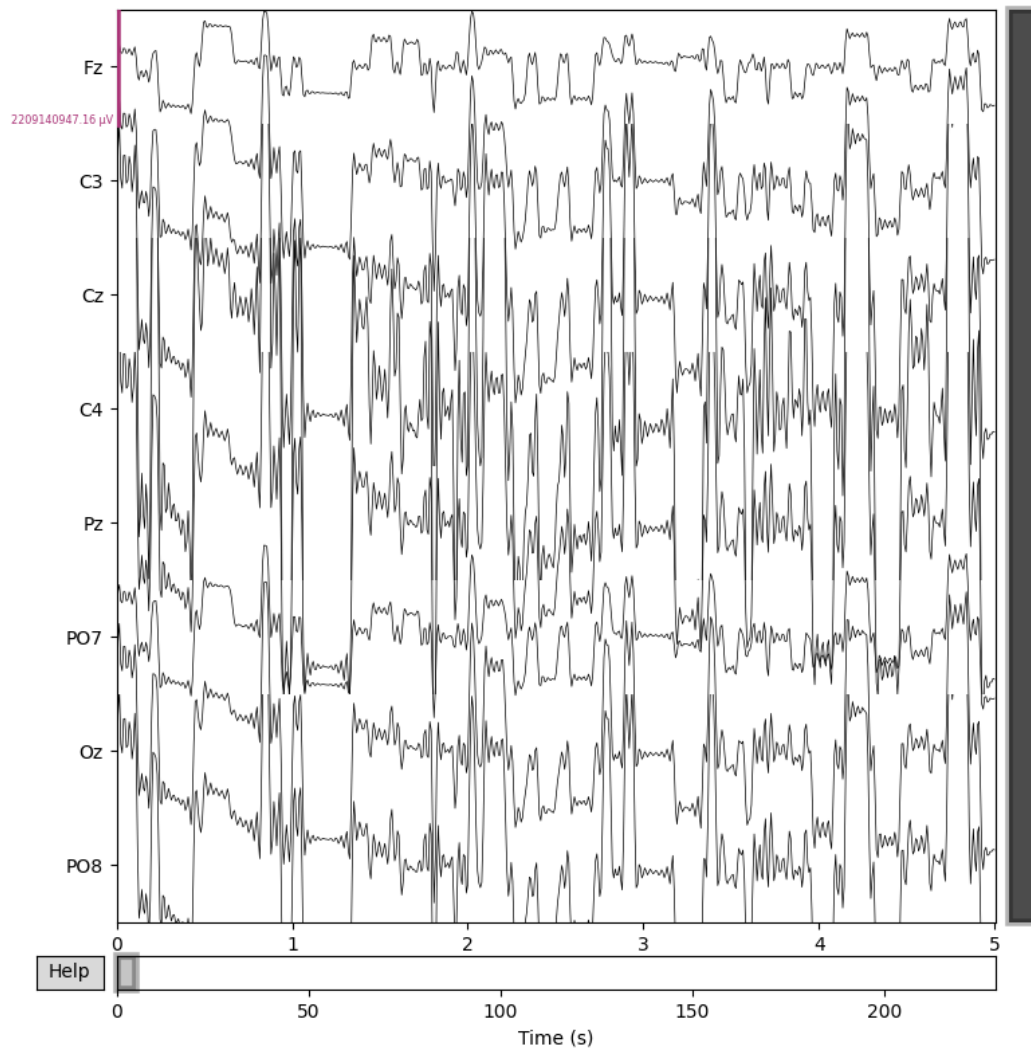
Opening raw data file 2. Loading & Digitization/EEG_CleanStream_09h_raw.fif...

Range : 0 ... 57254 = 0.000 ... 229.016 secs

Ready.

Reading 0 ... 57254 = 0.000 ... 229.016 secs...

2023-01-27 11:59:20+00:00



```
[ 'go', 'no-go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'no-go', 'go', 'no-go', 'no-go', 'go', 'no-g
o', 'no-go', 'go', 'no-go', 'no-go', 'no-go', 'no-go', 'no-go', 'no-go', 'no-go', 'no-go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go',
'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'go', 'no-go' ]
Annotations onsets: [ 4. 8.5 13. 17.5 22. 26.5 31. 35.5 40. 44.5 49. 53.5
58. 62.5 67. 71.5 76. 80.5 85. 89.5 94. 98.5 103. 107.5
112. 116.5 121. 125.5 130. 134.5 139. 143.5 148. 152.5 157. 161.5
166. 170.5 175. 179.5 184. 188.5 193. 197.5 202. 206.5 211. 215.5
220. 224.5 ]
2023-01-27 11:59:20+00:00 2023-01-27 11:59:24.000000
Writing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_05\Executive functions practical\4. Epoch\Annotated file
s\EEG_CleanStream_09h_annot_raw.fif
Closing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_05\Executive functions practical\4. Epoch\Annotated file
s\EEG_CleanStream_09h_annot_raw.fif
[done]
```

Step 3: epoching

```
In [20]: 1 from Epoching_erp import epoching
```

```
In [21]: 1 os.getcwd()
```

```
'C:\\Users\\IamAh\\Desktop\\UNIVERSITY - CSAI\\Cognitive Neuroscience\\Practical_05\\Executive functions practical'
```

First of all, you're doing great! Congratulations for getting this far, we are almost there!

In the next step, epoching, you will "epoch" your files: this means that the program will take each file and extract the 0.5-seconds long trials marked by the markers we inserted in the Annotations step and save this information in a new, "Epoch" file, which can be thought of as a N-dimensional numpy array of shape: trials(the amount of markers we inserted in the Annotations step) X nr_channels X time points.

In this case, we also add a baseline period while epoching, in order to compare and correct for brain activity **before** and **after** the trial onset.

In [22]:

```
1 for file in os.listdir("4. Epoch/Annotated files"):
2     if "raw" in file and file != '.ipynb_checkpoints__annot_raw.fif':
3         epoching(file, "4. Epoch/Annotated files")
```

Opening raw data file 4. Epoch/Annotated files/EEG_CleanStream_09h_annot_raw.fif...

Range : 0 ... 57254 = 0.000 ... 229.016 secs

Ready.

```
[ [ 1000  0  1]
[ 2125  0  2]
[ 3250  0  1]
[ 4375  0  1]
[ 5500  0  1]
[ 6625  0  1]
[ 7750  0  1]
[ 8875  0  1]
[10000  0  1]
[11125  0  1]
[12250  0  1]
[13375  0  1]
[14500  0  1]
[15625  0  1]
[16750  0  1]
[17875  0  2]
[19000  0  1]
[20125  0  2]
[21250  0  2]
[22375  0  1]
[23500  0  2]
[24625  0  2]
[25750  0  1]
[26875  0  2]
[28000  0  2]
[29125  0  2]
[30250  0  2]
[31375  0  2]
[32500  0  2]
[33625  0  2]
[34750  0  2]
[35875  0  1]
[37000  0  1]
[38125  0  1]
[39250  0  1]
[40375  0  1]
[41500  0  1]
[42625  0  1]
[43750  0  1]
[44875  0  1]
[46000  0  1]
[47125  0  1]
[48250  0  1]
[49375  0  1]
[50500  0  1]
[51625  0  1]
[52750  0  1]
[53875  0  1]
[55000  0  1]
[56125  0  2]]
```

{'go': 1, 'no-go': 2}

Not setting metadata

50 matching events found

Applying baseline correction (mode: mean)

0 projection items activated

Loading data for 50 events and 164 original time points ...

0 bad epochs dropped

Load the two files you epoched to make sure the time ranges are correct: the files should report: t = -152.00...500ms >> from -0.152 to 0.5 seconds.

Replace the '...' with the name of the epoched file

```
In [24]: 1 epoch_one = mne.read_epochs("4. Epoch/EEG_CleanStream_09h_epo.fif", preload=True)

Reading C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_05\Executive functions practical\4. Epoch\EEG_CleanStream_09h_epo.fif ...
Found the data of interest:
    t =   -152.00 ...    500.00 ms
    0 CTF compensation matrices available
Not setting metadata
50 matching events found
No baseline correction applied
0 projection items activated
```

Same for this epoch:

```
In [25]: 1 epoch_one.get_data().shape

(50, 8, 164)
```

Step 4: Event-related potential analysis (ERP)

The goal of ERP analysis is to plot across-trials averaged signal(s), where each plotted signal is the average of all the trials belonging to one specific class.

To do so, the first step is to create two **evoked** objects (evoked objects are objects that result from the averaging across specified trials within an epoch) for each epoched file; each evoked file will extract and average across one of the two trial classes (go/no-go).

```
In [26]: 1 go_one = epoch_one['go'].average()
2 no_go_one = epoch_one['no-go'].average()
```

In the next step, we create a dictionary where the key is the trial type and the value is a list containing the corresponding evoked objects, for each epoched file.

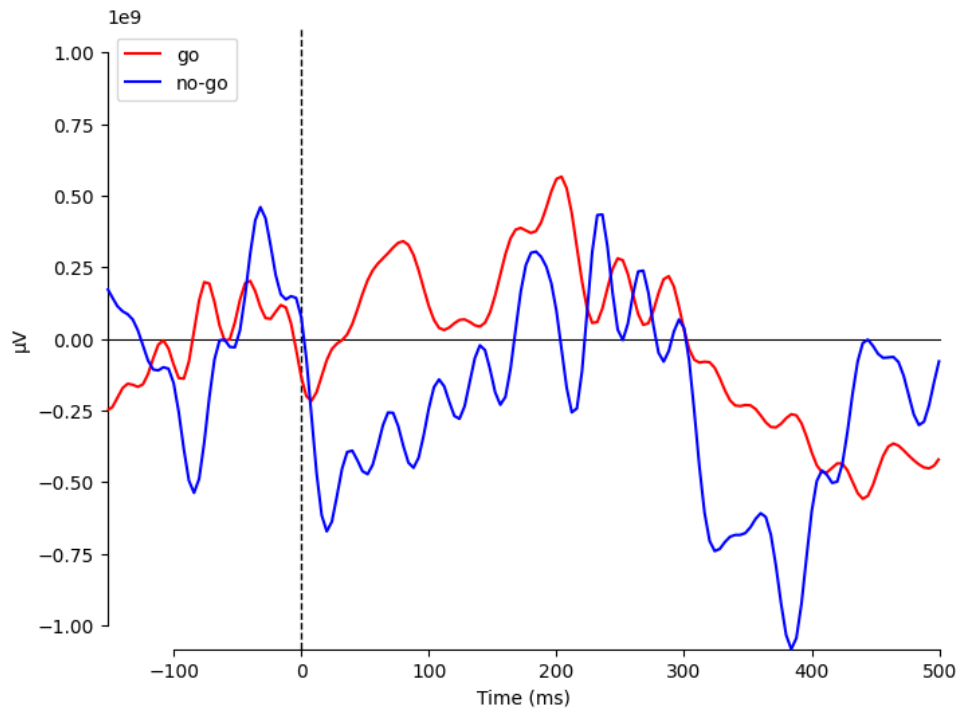
```
In [27]: 1 evoked = {"go": [go_one], "no-go": [no_go_one]}
```

Lastly, we plot the two ERP lines using a MNE function, to which we pass the following parameters:

1. the "evoked" dictionary
2. the method we want to use to combine the evoked objects
3. a color dictionary (color_dict) for the line plotting
4. a linestyle (linestyle_dict)
5. the time unit we want to visualise (ms are the standard for ERP plots)
6. the channels we would like to average across: since the P300 is known to peak in the posterior channels, we will pass such channels to this parameter, although feel free to tweak around!

```
In [28]: 1 color_dict = {'go':'red', 'no-go':'blue'}
2 linestyle_dict = {'go':'--', 'no-go':'-'}
3 mne.viz.plot_compare_evokeds(evoked, combine = "mean", colors=color_dict,
4                             linestyles=linestyle_dict, time_unit = "ms")
```

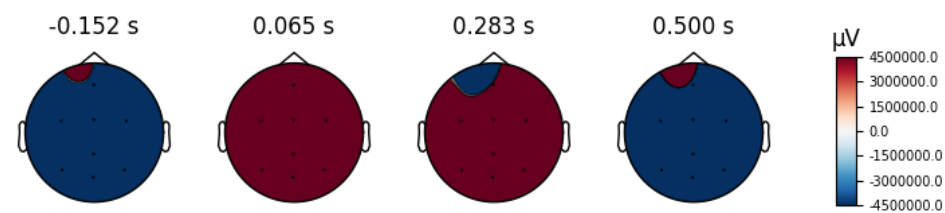
combining channels using "mean"
combining channels using "mean"



[<Figure size 800x600 with 1 Axes>]

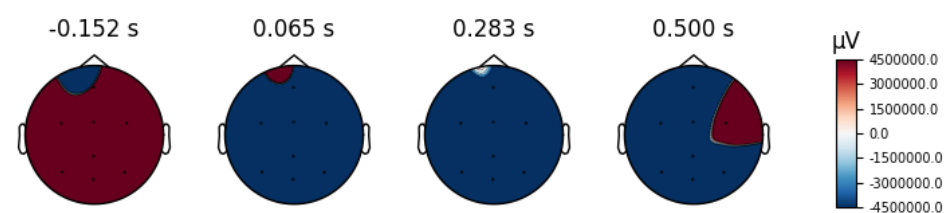
You can also, for each trial class, visualize a topomap, in order to inspect the buildup of brain activity throughout the averaged epochs.

```
In [29]: 1 mne.viz.plot_evoked_topomap((go_one), vlim = (-4500000, 4500000))
2 print("End")
```



End

```
In [30]: 1 mne.viz.plot_evoked_topomap((no_go_one), vlim = (-4500000, 4500000));
```



Step 5: Statistical analysis

In [31]:	1 from scipy import stats
In [32]:	1 go_evoked = [go_one] 2 3 go_grand_av = mne.grand_average(go_evoked) Identifying common channels ... <ipython-input-32-edc75f2e12bf>:3: RuntimeWarning: Only a single dataset was passed to mne.grand_average(). go_grand_av = mne.grand_average(go_evoked)
In [33]:	1 no_go_evoked = [no_go_one] 2 3 no_go_grand_av = mne.grand_average(no_go_evoked) Identifying common channels ... <ipython-input-33-e88ff527c565>:3: RuntimeWarning: Only a single dataset was passed to mne.grand_average(). no_go_grand_av = mne.grand_average(no_go_evoked)
In [34]:	1 stat, pvalue = stats.ttest_rel(go_grand_av.get_data(), no_go_grand_av.get_data())
In [35]:	1 print(pvalue) [7.12312123e-03 8.45471727e-03 8.95205382e-03 7.83232213e-03 5.67398150e-03 3.89946563e-03 2.68069880e-03 1.32828405e-03 1.08407563e-05 1.11916405e-01 4.23677988e-02 4.55566542e-02 6.69109775e-02 6.25190463e-02 2.13040196e-02 1.03758338e-02 8.46808472e-03 8.42426182e-03 9.01441866e-03 1.00020560e-02 1.14070508e-02 1.31828626e-02 1.10671299e-02 5.51677422e-01 2.17419513e-05 6.31875932e-04 2.66403090e-03 5.53762657e-03 5.14590241e-03 6.90002233e-03 8.72365612e-03 1.13147550e-02 1.64689312e-02 3.38231338e-02 1.55638341e-01 1.90228519e-01 4.27994527e-02 3.14036846e-02 4.40818194e-02 1.31997038e-01 2.72527934e-01 1.92586841e-06 2.16695246e-04 6.47112551e-04 9.05243364e-04 8.79420681e-04 6.76181466e-04 5.54773652e-04 6.90702560e-04 1.07677714e-03 1.54151977e-03 1.88867534e-03 2.00979440e-03 1.88525529e-03 1.60054388e-03 1.34881146e-03 1.30049422e-03 1.48069837e-03 1.80696715e-03 2.17153541e-03 2.48380070e-03 2.65685360e-03 2.55595818e-03 1.96932693e-03 9.15716080e-04 2.36714001e-04 1.47029916e-04 2.77047923e-04 4.80684906e-04 5.67576803e-04 3.92747502e-04 6.69261489e-05 3.19385663e-06 1.65469273e-02 1.41667673e-03 4.39473152e-07 6.65315601e-05 3.44763631e-04 6.50503948e-04 8.11425738e-04 7.39953409e-04 4.08157144e-04 6.35419039e-05 2.47453189e-05 2.54394588e-05 6.57145357e-06 6.68566926e-05 3.20908622e-04 8.44506252e-04 1.61122299e-03 2.54644883e-03 3.62812526e-03 5.04877214e-03 8.28378293e-03 5.11295010e-01 2.32887859e-03 3.80229626e-03 4.72014411e-03 5.74630384e-03 1.49557358e-02 1.03370960e-02 1.39426740e-02 3.16726722e-02 9.26880186e-01 3.70874816e-04 1.31762110e-03 2.56477934e-03 3.47552769e-02 6.49076970e-03 6.40904886e-03 7.85195934e-03 1.29865219e-02 8.35230911e-02 6.45467167e-01 2.37309023e-02 5.06139196e-04 7.55386483e-04 1.00029533e-03 1.14294195e-03 1.17219849e-03 1.06628288e-03 8.17465825e-04 5.02475228e-04 2.57397352e-04 1.26666975e-04 6.63214730e-05 3.70967091e-05 2.36124579e-05 2.43325355e-05 6.13125928e-05 2.34955175e-04 6.15232951e-04 1.01014806e-03 1.27479581e-03 1.41711400e-03 1.45818189e-03 1.36104537e-03 9.91508288e-04 2.99153686e-04 2.55146589e-02 8.58870689e-01 8.04030049e-01 1.22706276e-01 5.87767439e-02 9.64872681e-01 9.44945704e-02 2.66529908e-02 1.78160897e-02 1.74402898e-02 2.07092151e-02 2.70777136e-02 3.53008149e-02 4.03899928e-02 3.73269916e-02 3.01913838e-02 2.62461348e-02 2.90885450e-02 4.36462736e-02 7.86399528e-02 1.15672140e-01 1.02528467e-01 6.71662240e-02 4.65480938e-02 3.97754437e-02]
	The End!

